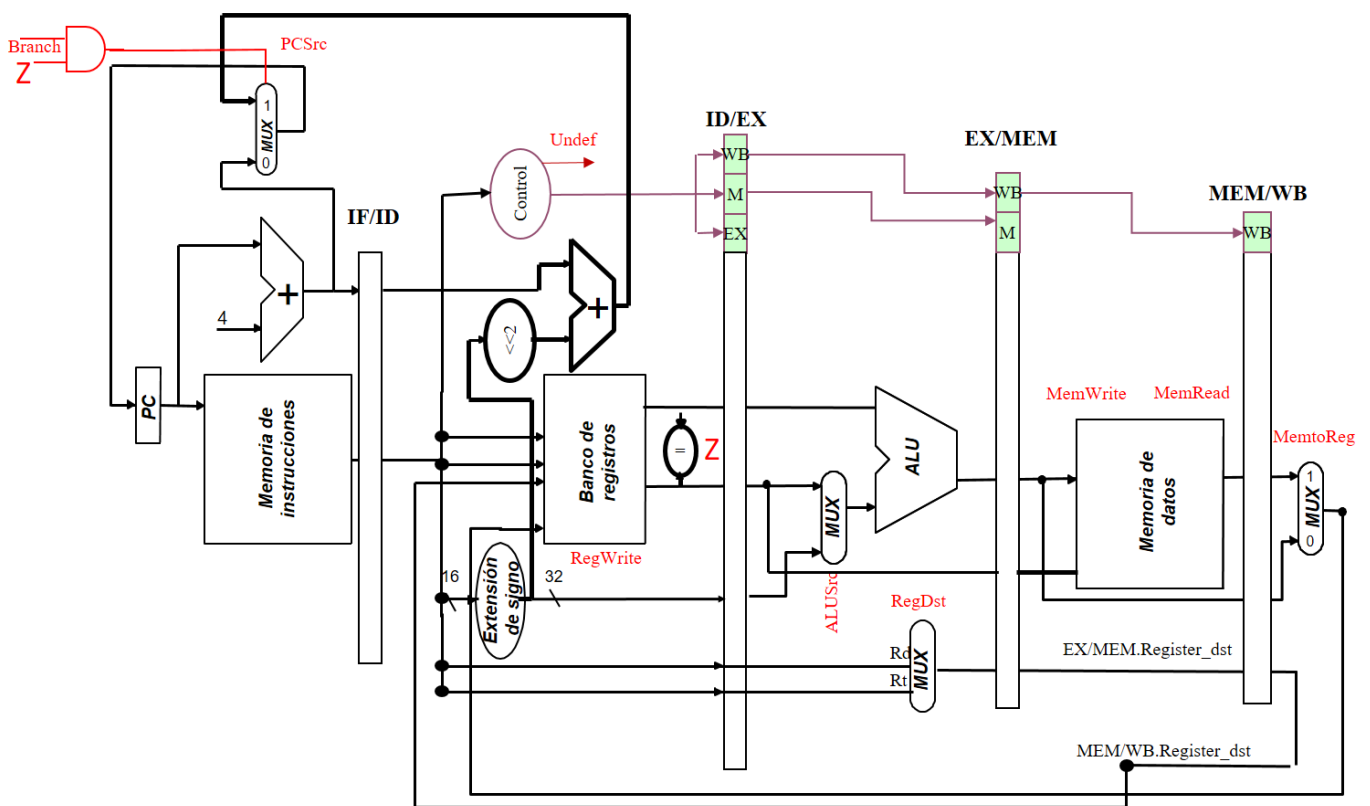


PROCESADOR SEGMENTADO PARA ALMA RETARDADA ADICIÓN DE INSTRUCCIONES

Práctica 3



Arquitectura y Organización de Computadores 2
2º Grado Ingeniería Informática

Luis M. Ramos
luisma@unizar.es

Alejandro Valero
alvabre@unizar.es

José Luis Briz
briz@unizar.es

Javier Resano
jresano@unizar.es



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

1 RESUMEN

La práctica consiste en realizar las modificaciones necesarias a un procesador segmentado sin detenciones descrito en VHDL para que ejecute dos nuevas instrucciones, diseñando un programa que compruebe todos los casos necesarios y verificando la ejecución mediante un simulador de VHDL.

Realizar esta práctica refuerza los conceptos de diseño del procesador segmentado estudiado en teoría y problemas, y proporciona experiencia en las técnicas de diseño mediante lenguajes de descripción de hardware (VHDL). Además, es el primer paso que deben de dar quienes opten por la vía de evaluación por proyecto, ya que será su primer contacto con el procesador base sobre el que habrán de realizar modificaciones posteriores más complejas.

Como **trabajo previo** a la sesión puedes realizar los apartados **2.1.a y 2.2**. Antes de empezar a trabajar conéctate a Moodle desde uno de los equipos del laboratorio y realiza el **control de asistencia a la sesión**.

La práctica finaliza cuando el procesador funciona correctamente y ha sido entregado a través de **Moodle**. También hay que entregar los **resultados del apartado 3**.

2 PROCESADOR SEGMENTADO PARA ALMA RETARDADA

2.1 ESTUDIO DEL PROCESADOR SEGMENTADO

- a) Descarga el código VHDL del procesador desde el recurso Moodle 'Código fuente VHDL práctica 3'. Estúdialo e intenta entender cómo funciona. Busca los componentes del procesador en el código e identifícalos con los componentes de las transparencias de clase. Es una buena idea hacerse una copia ampliada del MIPS y copiar los nombres de las señales usadas en VHDL para conectar y nombrar los componentes principales.

Observa la organización de las RAM: como vamos a usar programas pequeños utilizaremos memorias con 128 palabras de 32 bits. Las direcciones seguirán siendo de 32 bits, pero usaremos solo los 7 bits de menos peso.

- b) Este procesador no gestiona ningún tipo de riesgo, así que antes de ejecutar el programa de ejemplo tendrás que modificarlo **insertando las instrucciones nop adecuadas**. Recuerda actualizar la instrucción de salto, para que continúe saltando a la misma dirección que antes. Simula la ejecución del programa de ejemplo, mediante el flujo de trabajo con GHDL que ya conoces de la práctica anterior. Observa cómo se transmiten las señales de una etapa a otra atravesando los bancos de etapa. Comprueba que los registros y la memoria tienen el valor deseado. En Moodle hay un video que te ayudará a comenzar.

2.2 ADICIÓN DE INSTRUCCIONES

```
j1al rt, inmed16: BR(rt) ← PC + 4; PC ← PC + 4 + 4*SignExt(inmed16);  
ret rs: PC ← BR(rs);
```

- a) Dibuja sobre el esquema anterior las modificaciones necesarias para que el procesador pueda ejecutar las nuevas instrucciones y especifica las acciones RTL correspondientes.

El objetivo de estas instrucciones es saltar a una rutina y retornar a la dirección correcta al terminar su ejecución. Su implementación requiere modificar la lógica que actualiza el PC (entradas del mux y señales de control). También hay que gestionar la escritura del PC+4 en el banco de registros: hazla en la etapa WB, propagando PC+4 hasta la misma mediante las modificaciones necesarias. Tendrás que modificar también la UC para que reconozca las nuevas instrucciones y gestione sus señales de control correspondientes.

- b) Dibuja el formato de instrucción de MIPS que eliges para ambas instrucciones indicando el uso de sus campos.
- c) Utilizando las instrucciones anteriores escribe un programa ensamblador (*NIA.txt*) que usando un bucle cargue de memoria a registro los valores obtenidos en webdiis.unizar.es/~luisma/aoc2/νια2.php a partir de tu NIA, y calcule la suma de todos ellos, almacenándola finalmente en la @0. Inserta instrucciones `nop` para los retardos.

Debes utilizar al menos una subrutina, para aprovechar las instrucciones `jal` y `ret`. Recuerda que es una ALMA retardada y que por tanto todos los retardos son visibles al código en lenguaje máquina. En este procesador, RAM de datos y RAM de instrucciones responden en un ciclo.

- d) Prepara una segunda versión de tu programa (*NIA-R.txt*) reordenando el código mediante planificación estática, con el objetivo de minimizar el tiempo de ejecución.

2.3 DISEÑO EN VHDL

- a) Traslada las modificaciones que has pensado para implementar `jal` y `ret`. Añade un comentario con tu nombre e *email* de Unizar en la cabecera de todos los ficheros `vhd` que modifiques.
- b) Codifica las instrucciones de tus programas *NIA.txt* y *NIA-R.txt* en los ficheros de RAM de instrucciones *RAM_I_NIA.vhd* y *RAM_I_NIA-R.vhd*. Inicializa la RAM de datos (*RAM_NIA.vhd*) teniendo en cuenta lo indicado anteriormente.

Utiliza exclusivamente los códigos de operación siguientes y no otros:

<code>jal: 000101</code>	<code>ret: 000110</code>
--------------------------	--------------------------

Codificación de las operaciones de la ALU (campo `funct` y señal `ALU_ctrl`):

<code>+: 00000</code>	<code> -: 00001</code>	<code>AND: 00010</code>	<code>OR: 00011</code>
-----------------------	------------------------	-------------------------	------------------------

- c) Simula la ejecución y depura hasta que el programa funcione correctamente.
- d) Entrega a través de Moodle un fichero *NIA.zip* con los ficheros fuente completos y los ficheros *NIA.txt* y *NIA-R.txt*.

3 ANÁLISIS DE RENDIMIENTO

- a) Calcula el *speedup* de tu código optimizado usando primero el tiempo y luego la velocidad en MIPS. Explica las diferencias.
- b) A la vista de la expresión de descomposición del `tex` ($CPI \cdot N_i \cdot t_c$), ¿Cuál es el factor clave en la mejora en este caso?

